Lesson: Strings

CSE 4107 : Structured Programming I

Shahriar Ivan

Introduction

String Literals

- Strings are arrays of characters in which a special character—the null character—marks the end
- The C library provides a collection of functions for working with strings
- A ***string literal*** is a sequence of characters enclosed within double quotes:

```
"When you come to a fork in the road, take it"
```

- String literals may contain escape sequences
- Character escapes often appear in printf and scanf format strings

Introduction

String Literals

- For example, each \n character in the string

```
"Candy\nIs dandy\nBut liquor\nIs quicker.\n  --Ogden Nash\n"
```

- Causes the cursor to advance to the next line:

```
Candy
Is dandy
But liquor
Is quicker.ww
    --Ogden Nash
```

- Be careful about octal and hexadecimal escape sequences ⇒ Counted as one character

String Literals

Counting a String Literal

- The backslash character (\) can be used to continue a string literal from one line to the next:

```
printf("When you come to a fork in the road, take it.  \
    --Yogi Berra");
```

- In general, the \ character can be used to join two or more lines of a program into a single line

String Literals

Counting a String Literal

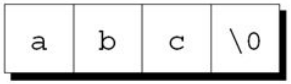
- There's a better way to deal with long string literals
- When two or more string literals are adjacent, the compiler will join them into a single string
- This rule allows us to split a string literal over two or more lines:

```
printf("When you come to a fork in the road, take it.  "
       "--Yogi Berra");
```

String Literals

How String Literals Are Stored

- *(n+1)* byte allocated for *n* length string
- 1 extra byte is used for ***null character*** ⇒ Denoting the end of string
- Null Character (\0) ⇒ a byte whose bits are all zeros
- The string literal "abc" is stored as an array of four characters:



- The string " " is stored as a single null character:



String Literals

How String Literals Are Stored

- Since a string literal is stored as an array, the compiler treats it as a pointer of type *char **
- Both printf and scanf expect a value of type *char ** as their first argument
- The following call of printf passes the address of *"abc"* (a pointer to where the letter a is stored in memory): {Not same as character constants}

```
printf("abc");

printf("\n");          // Legal ⇒ Pointer

printf('\n');          // WRONG ⇒ an integer
```

String Literals

Operations on String Literals

- String literals can be subscripted:

```
char ch, *p = "abc";
ch = "abc"[1];              // subscripted
```

The new value of ch will be the letter b

- A function that converts a number between 0 and 15 into the equivalent hex digit:

```
char digit_to_hex_char(int digit){
    return "0123456789ABCDEF"[digit];
}
```

String Literals

Operations on String Literals

- Attempting to modify a string literal causes undefined behavior:

```
char *p = "abc";
*p = 'd';              /*** WRONG ***/
```

- A program that tries to change a string literal may crash or behave erratically

String Variables

- Any *one-dimensional* array of characters can be used to store a string
- A string must be terminated by a ***null character***
- Difficulties with this approach:
  - It can be hard to tell whether an array of characters is being used as a string
  - String-handling functions must be careful to deal properly with the null character
  - Finding the length of a string requires searching for the null character

String Variables

- If a string variable holds 80 characters, it must be declared to have length 81:

```
#define STR_LEN 80
...
char str[STR_LEN+1];
```

- Adding 1 to the desired length allows room for the ***null character*** at the end of the string
- Defining a *macro* that represents 80 and then adding 1 separately is a common practice

String Variables

Initializing a String Variable

- A string variable can be initialized at the same time it's declared:

```
char pun1[9] = "pie rate";
```

p	i	e		r	a	t	e	\0

- Compiler automatically adds null character at the end
- Filled with extra null characters at the end if the input character size is less

```
char pun2[10] = "peas out";
```

p	e	a	s		o	u	t	\0	\0

String Variables

Initializing a String Variable

- Initializer cannot be longer than the variable, but can be the same length

```
char pun1[8] = "pie rate";
```

p	i	e		r	a	t	e

- Compiler automatically computes the length if its omitted
- Particularly useful if the initializer is long

```
char pun2[] = "peas out";
```

p	e	a	s		o	u	t

String Variables

Character Array vs Character Pointers

- The declaration:

```
char pun1[] = "pie rate";       // Declares pun1 to be an array

char *pun2 = "peas out";        // Declares pun2 to be a pointer
```

- Difference:

pun1[]	*pun2
Name of an array	Pointer to a string
Allocated space for string in memory	Points to the first memory location
Stored characters can be modified	Can’t be modified

String Variables

Character Array vs Character Pointers

- One possibility is to make p point to a string variable:

```
char str[STR_LEN+1], *p;
p = str;
```

- Another possibility is to make p point to a dynamically allocated string
- Using an uninitialized pointer variable as a string is a serious error

String Variables

Character Array vs Character Pointers

- An attempt at building the string "abc":

```
char *p;
p[0] = 'a';          /*** WRONG ***/
p[1] = 'b';          /*** WRONG ***/
p[2] = 'c';          /*** WRONG ***/
p[3] = '\0';         /*** WRONG ***/
```

- Since p hasn't been initialized, this causes undefined behavior

Reading and Writing Strings

- Writing a *string* is easy using either *printf* or *puts*
- Reading a *string* is a bit harder, because the input may be longer than the *string* variable into which it's being stored
- To read a *string* in a single step, we can use either *scanf* or *gets*
- As an alternative, we can read strings one character at a time

Reading and Writing Strings

Writing Strings Using printf and puts

- Using printf :
  - Format specifier: %-m.ps (Minus sign is optional)

```
char str[] = "yay, I'm in a string";
printf("%s\n", str);
```

  - Prints until null character is encountered
    - p ⇒ Number of characters to be displayed
    - m ⇒ Size of the string field
    - (-) ⇒ Left-justify within string field

```
printf("%4.5s\n", str);     // prints only the first four characters
```

Reading and Writing Strings

Writing Strings Using printf and puts

- Using puts:
  - The C library also provides puts:

    ```
    puts(str);
    ```

  - After writing a string, puts always writes an additional new-line character

Reading and Writing Strings

Writing Strings Using scanf and gets

- Using scanf :
  - Format specifier: %ns

    ```
    scanf("%s", str);
    ```

  - Treats str as a pointer ⇒ No & required
  - Skips white spaces, reads characters, stores them in str until white-space is encountered
  - Stores null character at the end
  - Doesn’t read a full line ⇒ Stops at new-line/space/tab

Reading and Writing Strings

Writing Strings Using scanf and gets

- Using gets :
  - Doesn’t skip white-space before starting
  - Reads until new-line character
  - Discards new-line and stores null character at the end

```
gets(str); // deprecated function - rather use fgets(char*, int, FILE*)
```

Reading and Writing Strings

Writing Strings Using scanf and gets

- Consider the following program fragment:

```
char sentence[STR_LEN+1];
scanf("%s", sentence);
```

- Suppose that after the prompt the user enters the line

Hello world! (denotes spaces)

Scanf will only store "Hello"

- If we replace scanf with gets(str),
- gets() will store " Hello world!"

Reading and Writing Strings

Writing Strings Using scanf and gets

- As they read characters into an array, scanf and gets have no way to detect when it's full
- Consequently, they may store characters past the end of the array, causing undefined behavior
- scanf can be made safer by the conversion specification %*n*s instead of %s
- *n* is an integer indicating the maximum number of characters to be stored
- gets is inherently unsafe; fgets is a much better alternative

Reading and Writing Strings

Writing Strings Using scanf and gets

- Programmers often write their own input functions.
- Issues to consider:
  - Should the function skip white space before beginning to store the string?
  - What character causes the function to stop reading: a new-line character, any white-space character, or some other character? Is this character stored in the string or discarded?
  - What should the function do if the input string is too long to store: discard the extra characters or leave them for the next input operation?

Reading and Writing Strings

Reading Strings Character by Character

- Suppose we need a function that:
  - doesn't skip white-space characters,
  - stops reading at the first new-line character (which isn't stored in the string),
  - discards extra characters
- A prototype for the function:

```
int read_line(char str[], int n);
```

- If the input line contains more than n characters, read_line will discard the additional characters
- read_line will return the number of characters it stores in str

Reading and Writing Strings

Reading Strings Character by Character

- read_line consists primarily of a loop that calls getchar to read a character and then stores the character in str, provided that there's room left:

```
int read_line(char str[], int n){
    int ch, i = 0;
    while ((ch = getchar()) != '\n')   // ends at newline
        if (i < n)
            str[i++] = ch;
    str[i] = '\0';                     /* terminates string */
    return i;                          /* number of characters stored */
}
```

Accessing the Characters in a String

- A function that counts the number of spaces in a string: {Array subscripting}

```
int count_spaces(const char s[]){
    int count = 0, i;
    for (i = 0; s[i] != '\0'; i++)
        if (s[i] == ' ')
            count++;
    return count;
}
```

Accessing the Characters in a String

- A version that uses pointer arithmetic instead of array subscripting :

```
int count_spaces(const char *s){
        int count = 0;
        for (; *s != '\0'; s++)
                if (*s == ' ')
                        count++;
        return count;
}
```

Accessing the Characters in a String

- *s can be modified despite being const ⇒ Copy of pointer
  - Ensures we don’t modify the string
- Which one is better? ⇒ Can use either. Traditionally we use pointer
- String parameter in function? ⇒ Can be both. No difference
- Argument passed for both parameters? ⇒ Array name, Pointer variable, or string literal

Using the C String Library

- Can’t use operators to copy, compare, or concatenate strings ⇒ as Arrays

```
char s1[10], s2[10];
…
s1 = "abc";          // Wrong
s2 = s1;             // Wrong
if(s1 == s2) …       // Wrong
```

- Initialization using “=” is legal

```
char s1[10] = "abc";
```

Using the C String Library

- “==” compares pointers in this case
- The C library provides a rich set of functions for performing operations on strings
- Programs that need string operations should contain the following line:

```
#include <string.h>
```

- In subsequent examples, assume that str1 and str2 are character arrays used as strings

Using the C String Library

The strcpy Function

- Prototype for the strcpy function:

```
char *strcpy(char *s1, const char *s2);
```

- strcpy copies the string s2 into the string s1.
  - To be precise, we should say “strcpy copies the string pointed to by s2 into the array pointed to by s1.”
- strcpy returns s1 (a pointer to the destination string)

Using the C String Library

The strcpy Function

- A call of strcpy that stores the string "abcd" in str2:

```
strcpy(str2, "abcd");          /* str2 now contains "abcd" */
```

- A call that copies the contents of str2 into str1:

```
strcpy(str1, str2);            /* str1 now contains "abcd" */
```

- In the call strcpy(str1, str2), strcpy has no way to check that the str2 string will fit in the array pointed to by str1
- If it doesn't, undefined behavior occurs

Using the C String Library

The strcpy Function

- Solution: Use strncpy(s1, s2, sizeof(s1))

```
// makes sure string ends with \0

strncpy(s1, s2, sizeof(s1) - 1);

s1[sizeof(s1) - 1] = '\0';
```

Using the C String Library

The strlen Function

- Prototype

```
size_t strlen(const char *s);
```

- size_t ⇒ Defined as one of the unsigned integer types
- Returns length of s excluding the null character

```
int len;
len = strlen("abc");          // len is now 3
len = strlen("");             // len is now 0
strcpy(s1, "abcd");
len = strlen(s1);             // len is now 4
```

- Doesn’t measure the length of the variable, only the stored string

Using the C String Library

The strcat Function

- Prototype:

```
char *strcat(char *s1, const char *s2);
```

- Appends the contents of s2 at the end of s1
- Return pointer to s1

```
strcpy(s1, "abc");
strcat(s1, "def");          // s1 contains "abcdef"
strcpy(s2, "ghi");          // s2 contains "ghi"
strcat(s1, s2);             // s1 contains "abcdefghi"
```

- Doesn't check if array is long enough ⇒ Undefined behavior
- Solution: Use strncat ⇒ `strncat(s1, s2, sizeof(s1) - strlen(s1) - 1);`

Using the C String Library

The strcmp Function

- Prototype

```
int strcmp(const char *s1, const char *s2);
```

- Compares s1 and s2
- Returns
  - 0 ⇒ If s1 and s2 are equal
  - Less than 0 ⇒ If s1 is less than s2
  - Greater than 0 ⇒ If s1 is greater than s2

```
// Checks if s1 is less than or equal to s2

if (strcmp(s1, s2) <= 0) ...
```

Using the C String Library

The strcmp Function

- As it compares two strings, strcmp looks at the numerical codes for the characters in the strings
- Some knowledge of the underlying character set is helpful
- Important properties of ASCII:
  - A–Z, a–z, and 0–9 have consecutive codes
  - All upper-case letters are less than all lower-case letters
  - Digits are less than letters
  - Spaces are less than all printing characters

Programming Task

Printing a One-Month Reminder List

- The program prints a one-month list of daily reminders
- The user will enter a series of reminders, with each prefixed by a day of the month
- When the user enters 0 instead of a valid day, the program will print a list of all reminders entered, sorted by day

Programming Task

Printing a One-Month Reminder List

```
Enter day and reminder: 24 Susan's birthday
Enter day and reminder: 5 6:00 - Dinner with Marge and Russ
Enter day and reminder: 26 Movie - "Chinatown"
Enter day and reminder: 7 10:30 - Dental appointment
Enter day and reminder: 12 Movie - "Dazed and Confused"
Enter day and reminder: 0

Day Reminder
  5 6:00 - Dinner with Marge and Russ
  7 10:30 - Dental appointment
 12 Movie - "Dazed and Confused"
 24 Susan's birthday
 26 Movie - "Chinatown"
```

Array of Strings

- There is more than one way to store an array of strings
- One option is to use a two-dimensional array of characters, with one string per row:

```
char planets[][8] = {"Mercury", "Venus", "Earth", "Mars", "Jupiter", "Saturn",
                     "Uranus", "Neptune", "Pluto"};
```

- The number of rows in the array can be omitted, but we must specify the number of columns

Array of Strings

- Unfortunately, the planets array contains a fair bit of wasted space (extra null characters):

	0	1	2	3	4	5	6	7
0	M	e	r	c	u	r	y	\0
1	V	e	n	u	s	\0	\0	\0
2	E	a	r	t	h	\0	\0	\0
3	M	a	r	s	\0	\0	\0	\0
4	J	u	p	i	t	e	r	\0
5	S	a	t	u	r	n	\0	\0
6	U	r	a	n	u	s	\0	\0
7	N	e	p	t	u	n	e	\0
8	P	l	u	t	o	\0	\0	\0

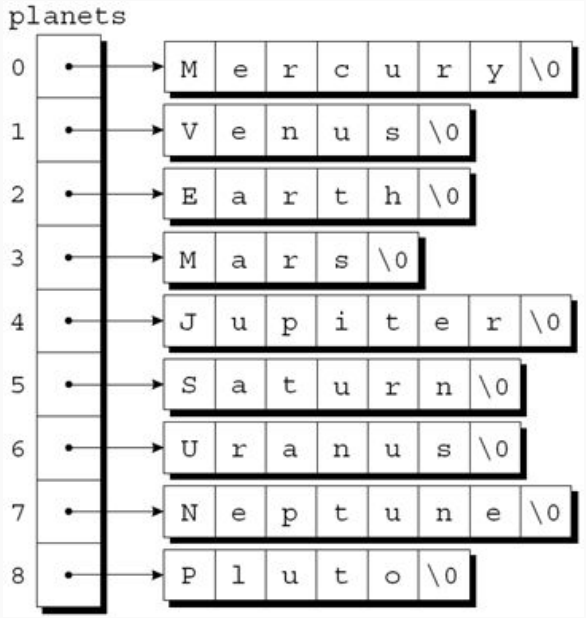
Array of Strings

- Most collections of strings will have a mixture of long strings and short strings.
- What we need is a ***ragged array,*** whose rows can have different lengths.
- We can simulate a ragged array in C by creating an array whose elements are *pointers* to strings:

```
char *planets[] = {"Mercury", "Venus", "Earth", "Mars", "Jupiter", "Saturn",
                   "Uranus", "Neptune", "Pluto"};
```

Array of Strings

- This small change has a dramatic effect on how planets is stored:



Array of Strings

- To access one of the planet names, all we need do is subscript the planets array
- Accessing a character in a planet name is done in the same way as accessing an element of *two-dimensional* array
- A loop that searches the planets array for strings beginning with the letter M:

```
for (i = 0; i < 9; i++)
    if (planets[i][0] == 'M')
        printf("%s begins with M\n", planets[i]);
```

Array of Strings

Command-Line Arguments

- Command-line information is available to all programs, not just operating system commands
- To obtain access to ***command-line arguments***, main must have two parameters:

```
int main(int argc, char *argv[]){
  …
}
```

- Command-line arguments are called ***program parameters*** in the C standard

Array of Strings

Command-Line Arguments

- argc (“argument count”) is the number of command-line arguments
- argv (“argument vector”) is an array of pointers to the command-line arguments (stored as strings)
- `argv[0]` points to the name of the program, while `argv[1]` through `argv[argc-1]` point to the remaining command-line arguments
- `argv[argc]` is always a ***null pointer***—a special pointer that points to nothing
  - The macro NULL represents a null pointer

Array of Strings

Command-Line Arguments

```
#include<stdio.h>
int main(int argc,char* argv[]){
    int counter;
    printf("Program Name Is: %s",argv[0]);
    if(argc==1)
        printf("\nNo Extra Command Line Argument Passed Other Than Program Name");
    if(argc>=2){
        printf("\nNumber Of Arguments Passed: %d",argc);
        printf("\n----Following Are The Command Line Arguments Passed----");
        for(counter=0;counter<argc;counter++)
            printf("\nargv[%d]: %s",counter,argv[counter]);
    }
    return 0;
}
```

Array of Strings

Command-Line Arguments

```
$ ./a.out First Second Third
Program Name Is: ./a.out

Number Of Arguments Passed: 4

----Following Are The Command Line Arguments Passed----
argv[0]: ./a.out
argv[1]: First
argv[2]: Second
argv[3]: Third
```

Array of Strings

Command-Line Arguments

- Can be accessed using loop

```
int i;
for (i = 1; i < argc; i++)
        printf("%s\n", argv[i]);
```

Programming Task

Checking Planet Names

- The program is to check a series of strings to see the names of planets
- The strings are put on the command line:

```
planet Jupiter venus Earth fred
```

- The program will indicate whether each string is a planet name and display the planet's number:

```
Jupiter is planet 5
venus is not a planet
Earth is planet 3
fred is not a planet
```